

Deep Learning - Theory and Practice

Linear Regression, Least Squares
Classification and Logistic Regression

27-02-2020

<http://leap.ee.iisc.ac.in/sriram/teaching/DL20/> → Assignment 2
→ Exam-1.

deeplearning.cce2020@gmail.com



Midterm Exam - 2.

→ March 5 (6:00pm)

1 hour

In class for online

→ Open book, notes etc
(no electronics)

→ MCQ but with reasoning

[Till 8pm on
27-2-2020]

→ Absence on 5th [Reason by email]

→ Separate exam.

Logistic Regression

- ❖ 2- class logistic regression

$$p(C_1|\phi) = y(\phi) = \sigma(\mathbf{w}^T \phi)$$

- ❖ Maximum likelihood solution

$$\nabla E(\mathbf{w}) = \sum_{n=1}^N (y_n - t_n) \phi_n$$

- ❖ K-class logistic regression

$$p(C_k|\phi) = y_k(\phi) = \frac{\exp(a_k)}{\sum_j \exp(a_j)}$$

- ❖ Maximum likelihood solution

$$a_k = \mathbf{w}_k^T \phi.$$

$$\nabla_{\mathbf{w}_j} E(\mathbf{w}_1, \dots, \mathbf{w}_K) = \sum_{n=1}^N (y_{nj} - t_{nj}) \phi_n$$

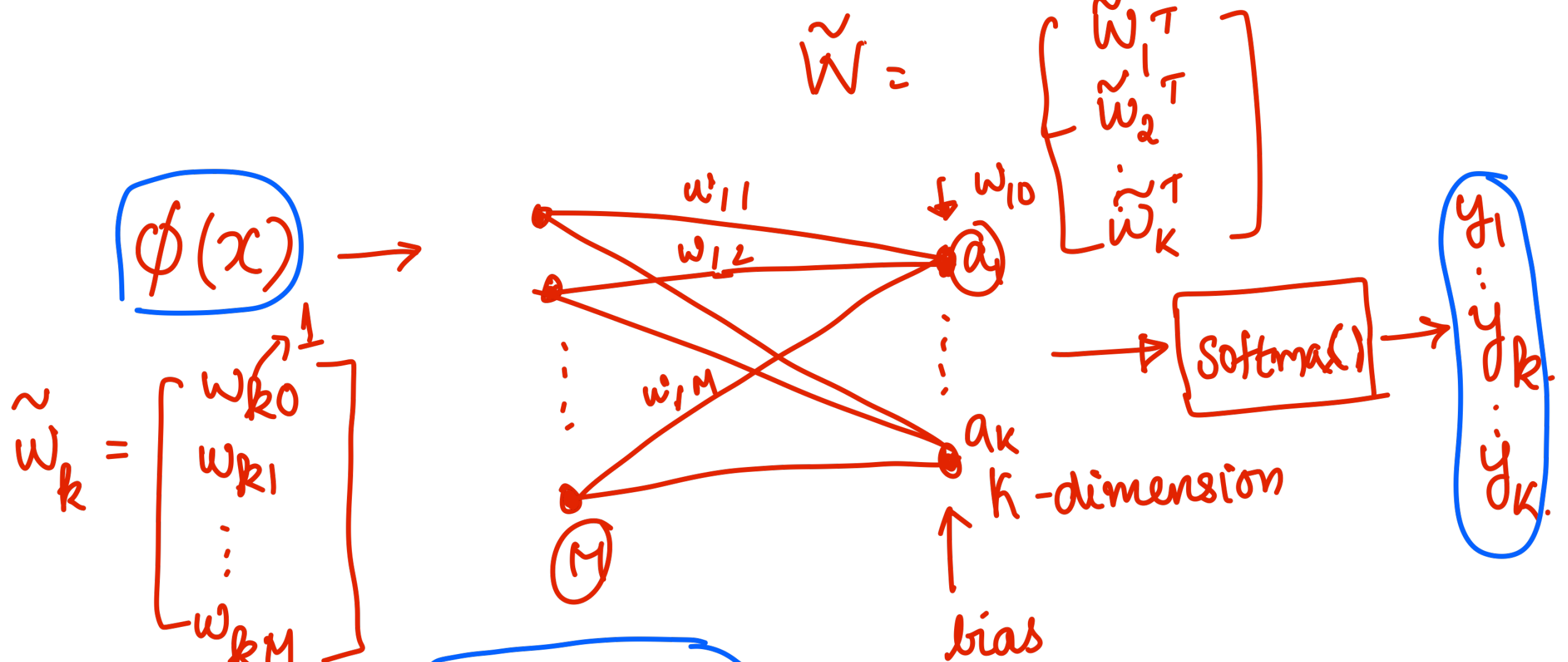
Logistic Regression ($k \geq 2$)

$$\boxed{P(C_k | \phi(x))} = \frac{p(\phi(x)/C_k) P(C_k)}{\sum_{j=1}^k p(\phi(x)/C_j) P(C_j)}$$

$$\log P(\phi(x), C_k) = a_k = \tilde{w}_k^T \tilde{\phi}(x)$$

Approximation

$$P(C_k | \phi(x)) = \frac{\exp(a_k)}{\sum_{j=1}^k \exp(a_j)} = \underline{\underline{\text{softmax}(a_k)}}$$



Properties of $y_k = \frac{e^{a_k}}{\sum_j e^{a_j}}$

$\phi(x) \in \mathbb{R}^M$

$a_k = \tilde{w}_k^T \phi(x)$

(i) $\sum_k y_k = 1$

(ii) $0 \leq y_k \leq 1 ; \forall k.$

MNIST $M = 784 (28^2)$

$K = 10$

Training

$$\tilde{W} = [\tilde{w}_1 \dots \tilde{w}_k]$$

Likelihood

t_n as one-hot encoding $\in \mathbb{R}^k$

$$T = [t_1 \dots t_N]$$

$$P(T / \tilde{w}_1, \dots, \tilde{w}_k) = \prod_{n=1}^N \prod_{k=1}^k \left(\underbrace{P C_k / \phi(x_n)}_{y_k(\phi(x_n))} \right)^{t_{nk}}$$

$$\frac{\partial L}{\partial w_j} =$$

$$-\sum_{n=1}^N \left(\underbrace{y_{nj}}_{\text{true label}} - \underbrace{t_{nj}}_{\text{predicted label}} \right) \phi(x_n)$$

Cross entropy
- $\log p(T/...)$

Soln is derived in an iterative fashion

$$\tilde{W}^{t+1} = \tilde{W}^t - \eta \frac{\partial L}{\partial \tilde{W}} \bigg|_{\tilde{W} = \tilde{W}^t} \quad [\text{Gradient Descent}]$$

Improving Learning $\rightarrow$ Batchwise Training
 $\rightarrow$ Stochastic Gradient Descent (SGD)

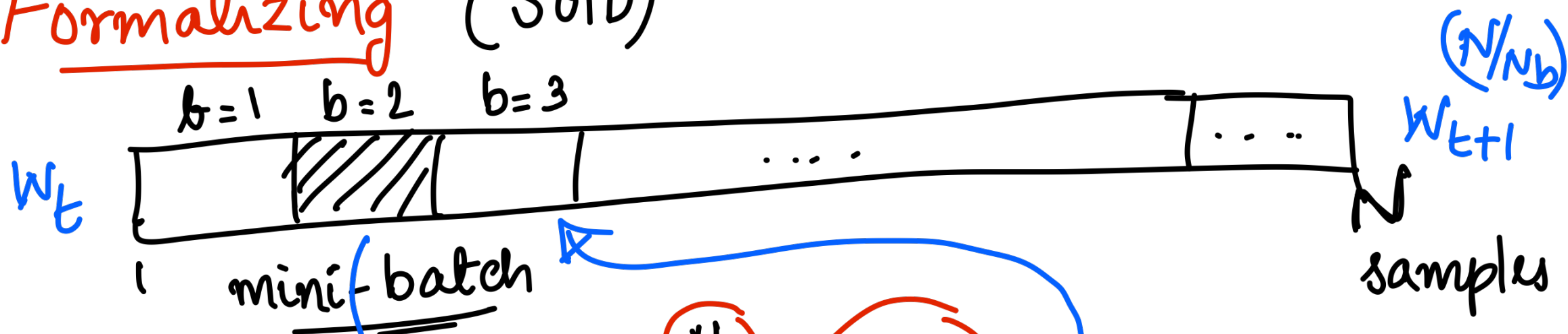
Gradient $\rightarrow \frac{\partial L}{\partial W}$ is used $-\eta \times \frac{\partial L}{\partial W}$
All samples are ind. $\rightarrow \sum_{n=1}^N \frac{\partial L_n}{\partial W} \rightarrow \frac{1}{N} \sum_{n=1}^N \frac{\partial L_n}{\partial W}$

and take factor N into l.r.

Can we approximate average with smaller
sample size $N \rightarrow N_b$ [Batch]

Typical batch size 128, 64, (few hundreds).
 $\rightarrow$ Trade off between speed v/s accuracy.

Formalizing (SGD)

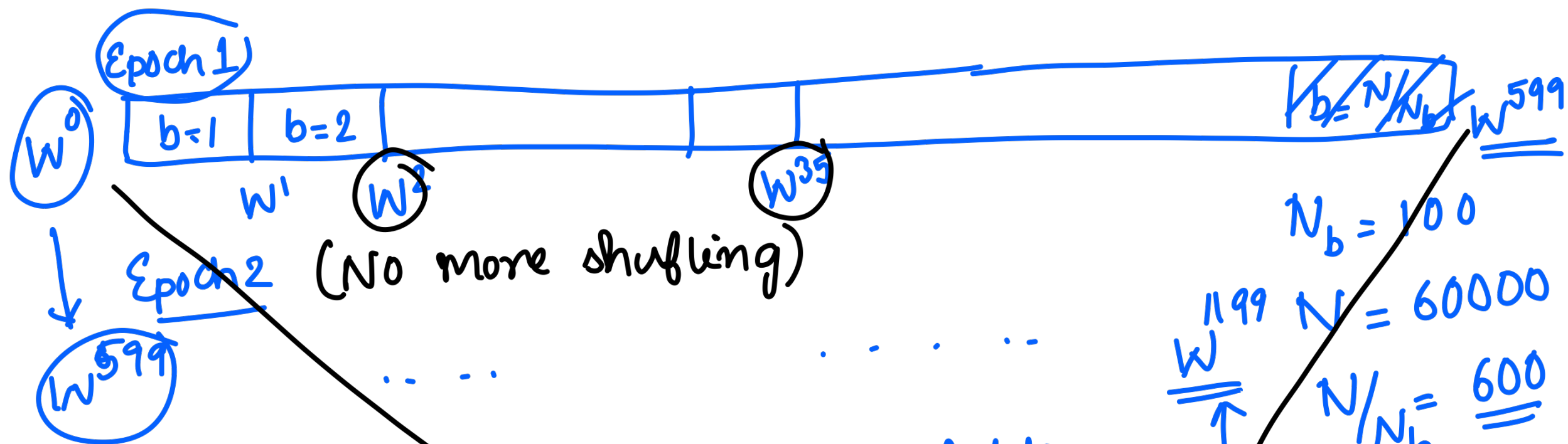


$$W^{t+1} = W^t - \eta \frac{1}{N_b} \sum_{n=1}^{N_b} \left. \frac{\partial L_n}{\partial W} \right|_{W=W^t} \rightarrow W^{t+1}$$

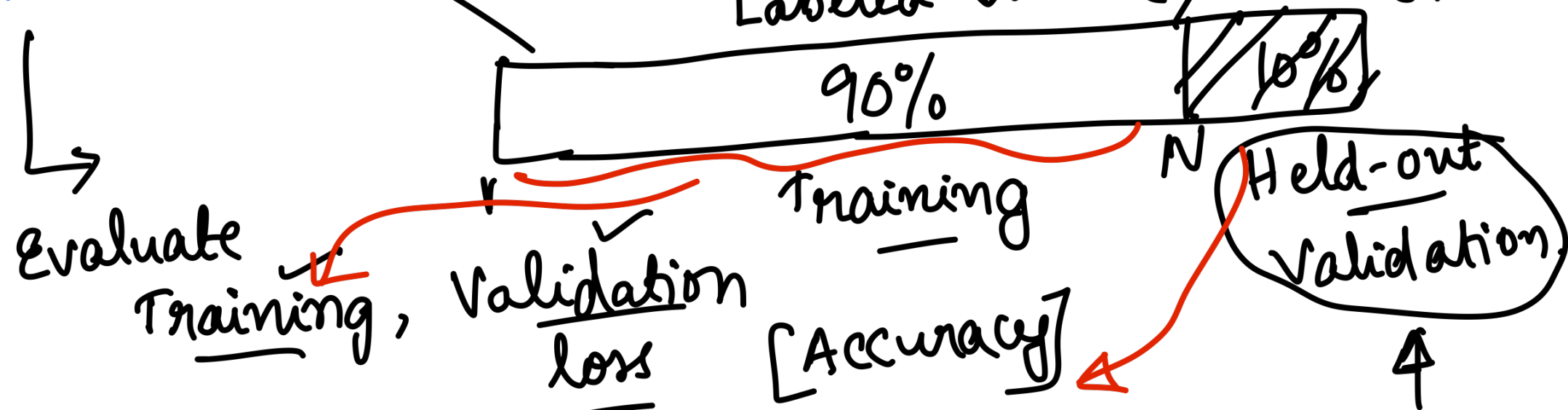
$N_b = 100$
 For eg:
 $N = 60,000$

Assumption

- * Each mini batch is a good representation of full batch
- * Random shuffling of data before choosing mini-batch
- * One run over all minibatches $\rightarrow$ iteration / epoch.

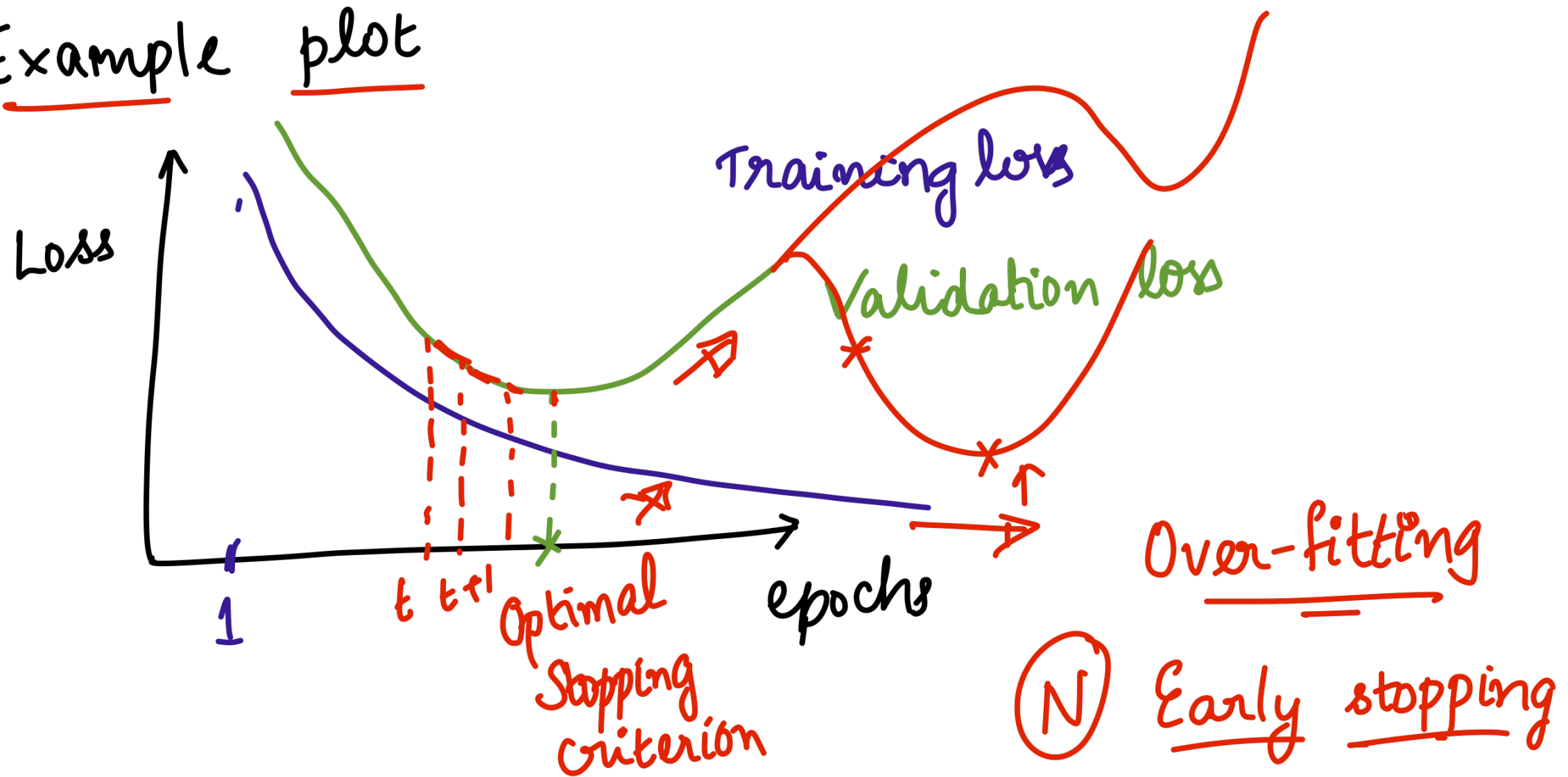


* After each epoch, model validation
Labeled Data (Shuffling)

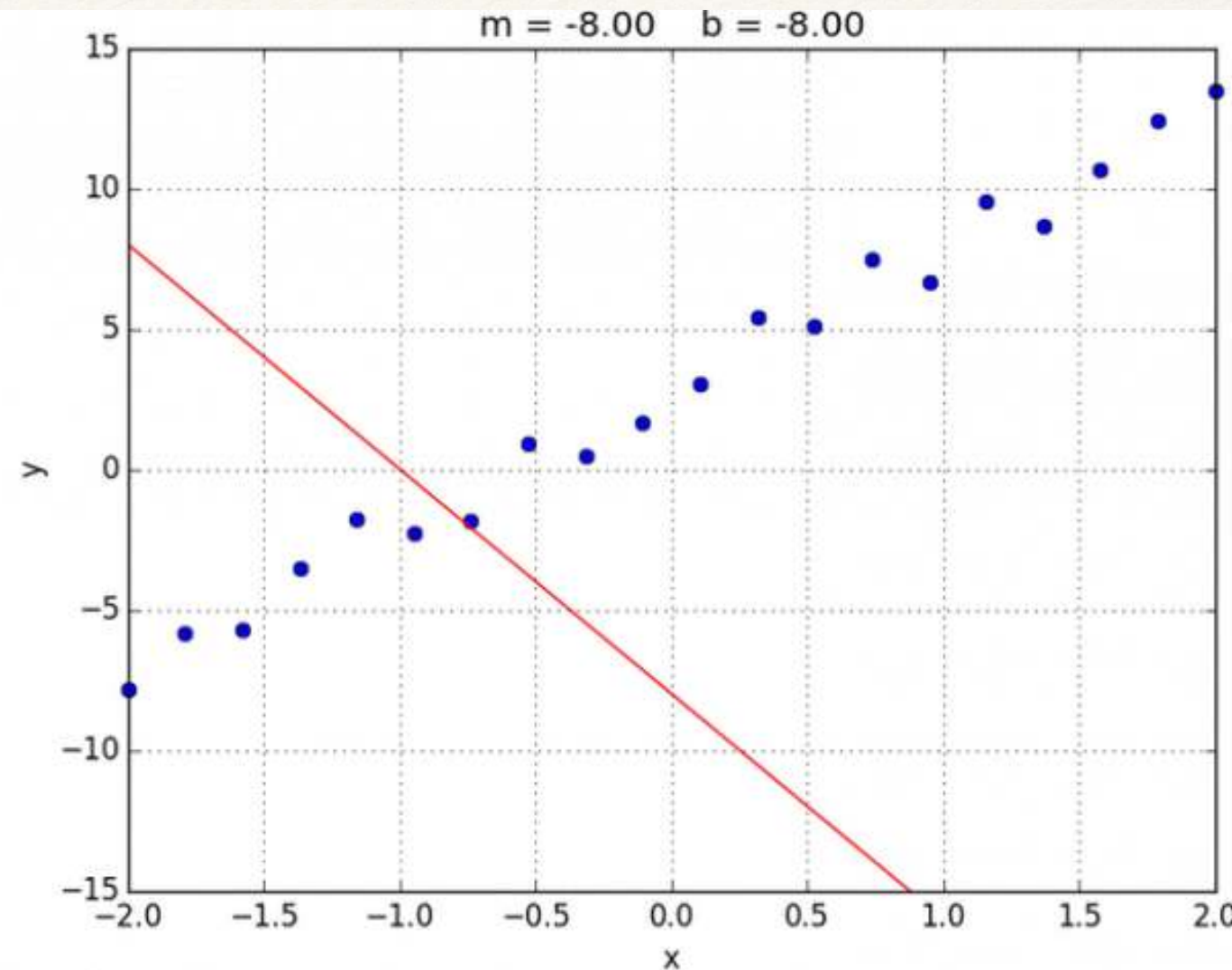
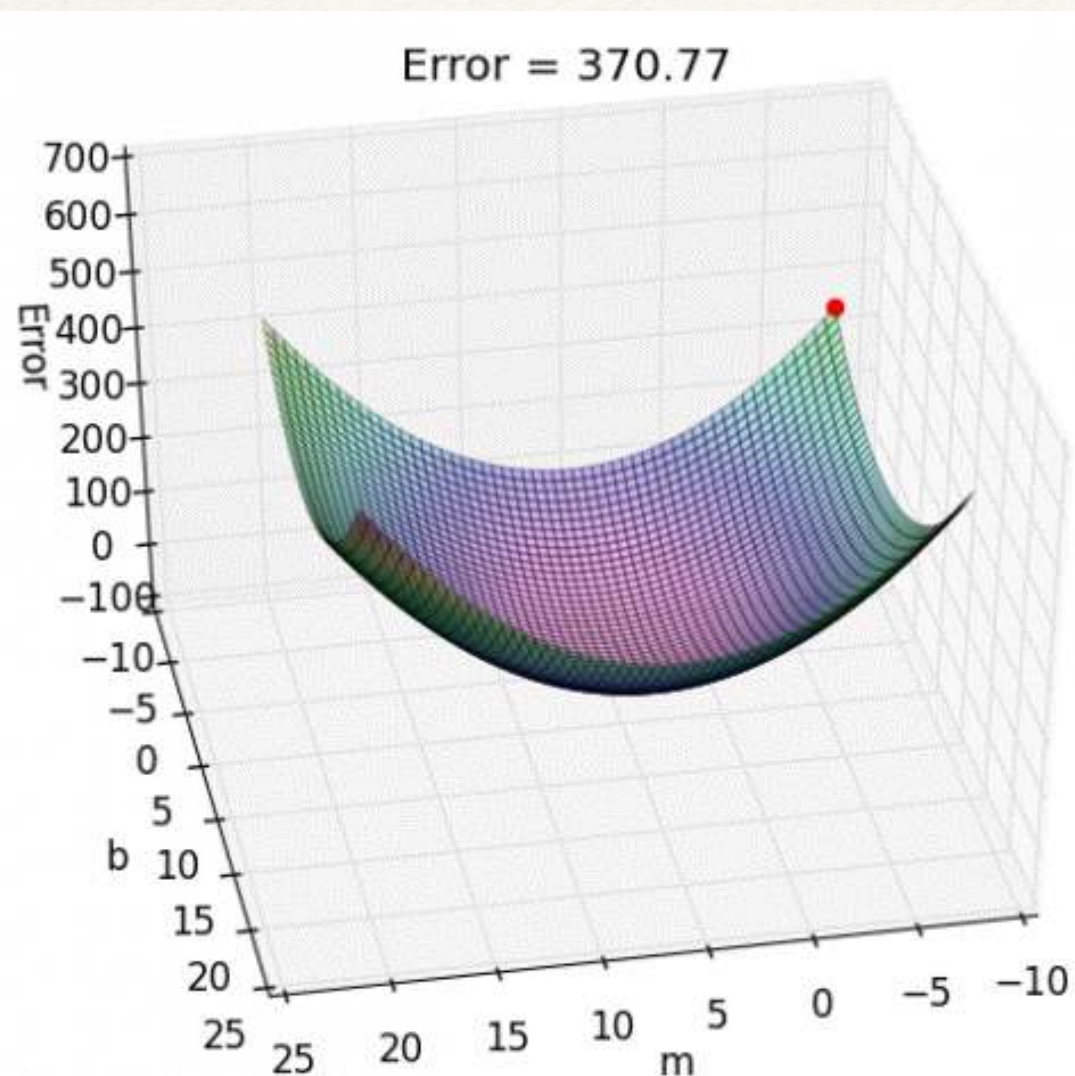


For ex: Cross entropy loss.

Example plot



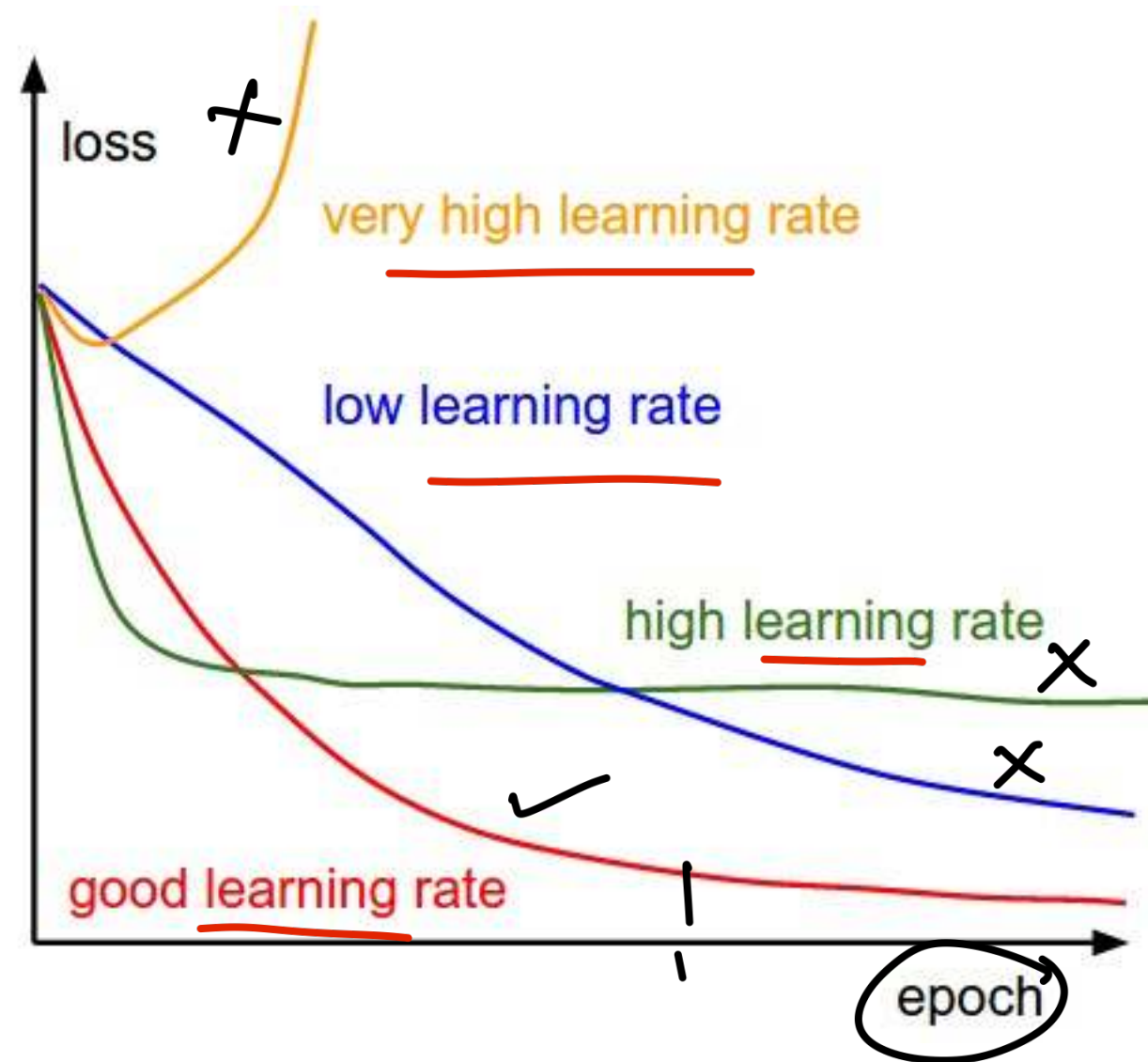
Learning Using Gradient Descent



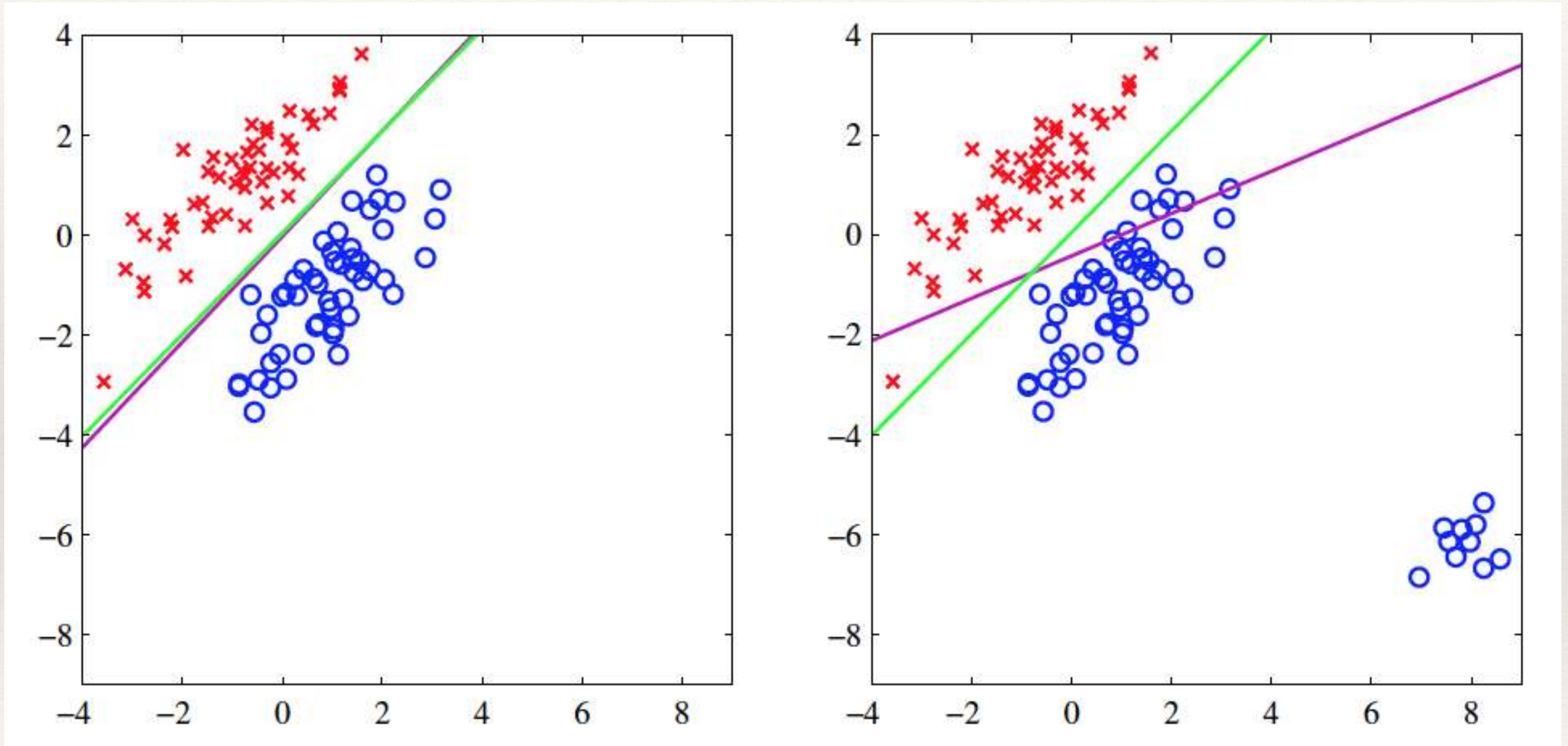
Assignment — Back propagation code ^{it} by
hand. $\phi(x) = \underline{\underline{x}}$

Parameter Learning

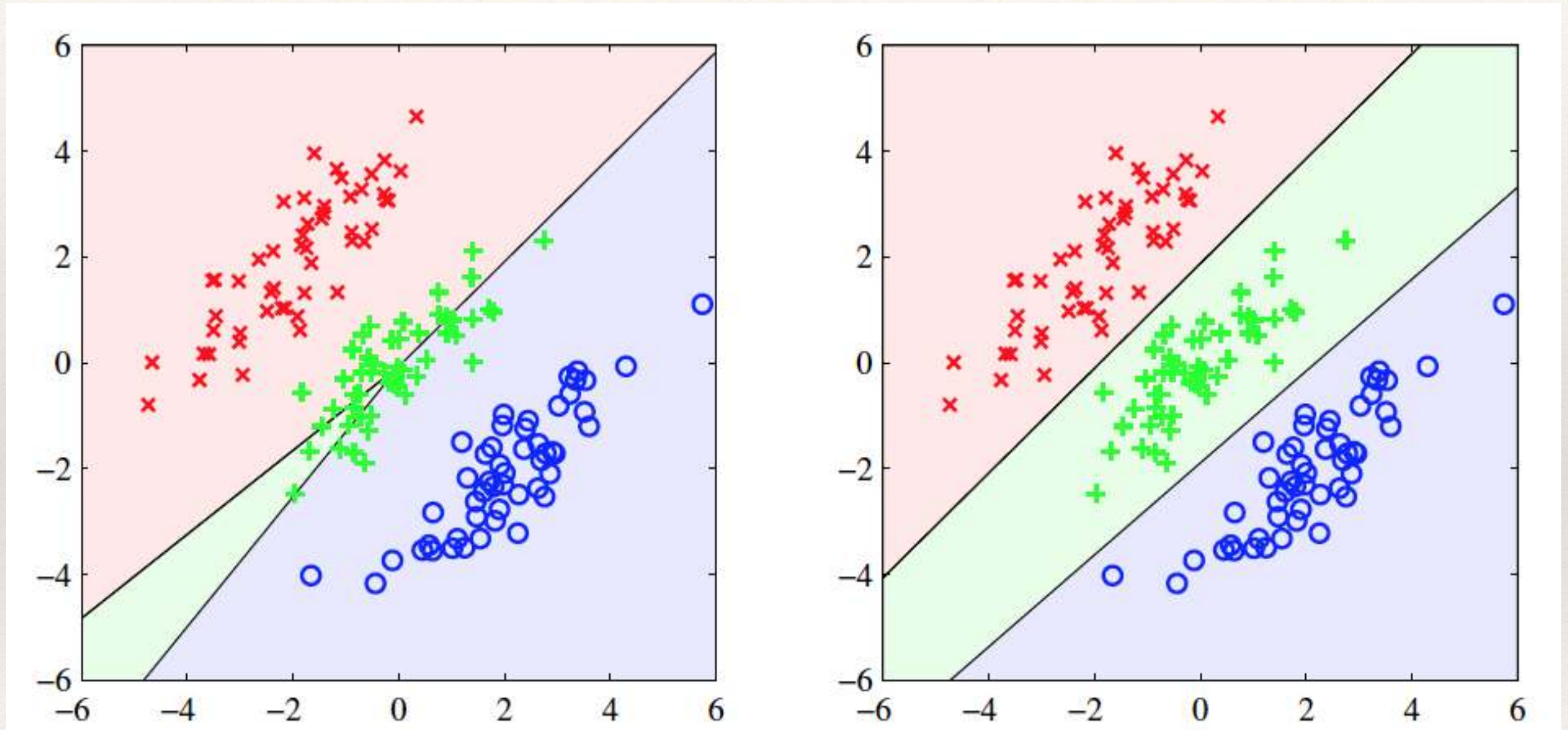
- Solving a non-convex optimization.
- Iterative solution.
- Depends on the initialization.
- Convergence to a local optima.
- Judicious choice of learning rate



Least Squares versus Logistic Regression

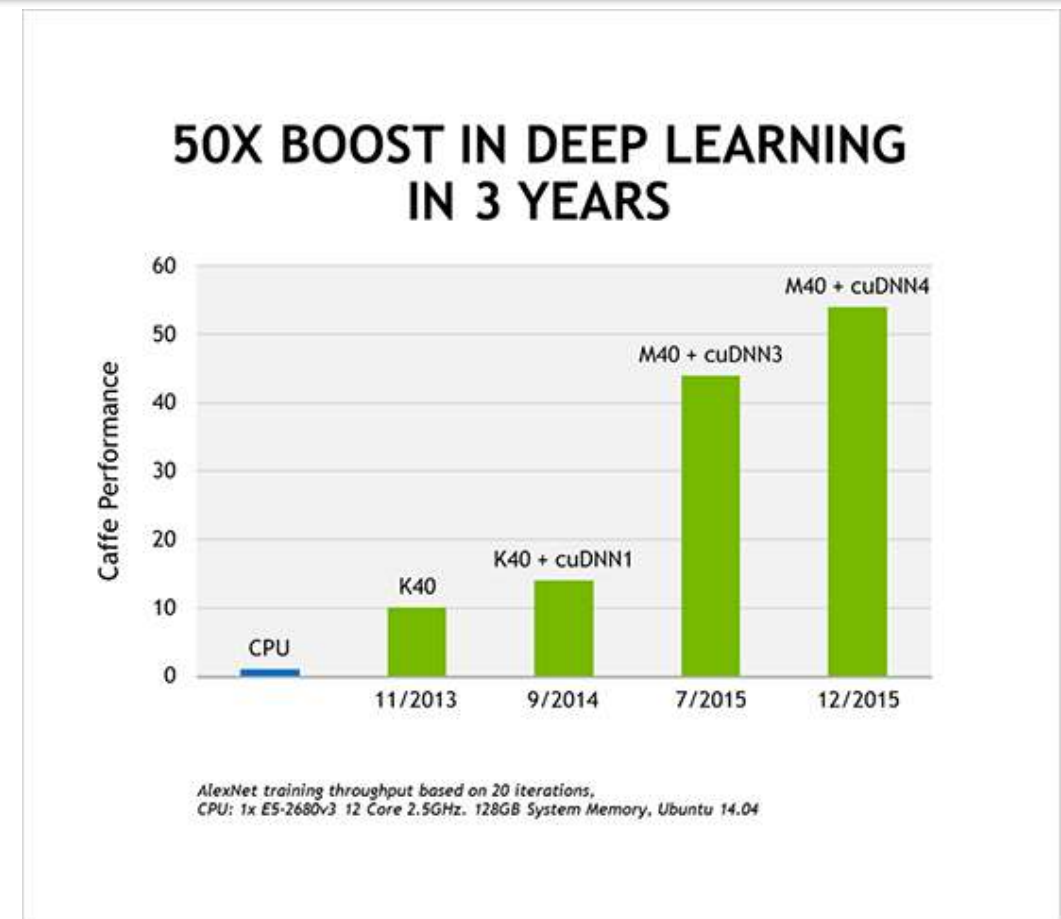


Least Squares versus Logistic Regression



Bishop - PRML book (Chap 4)

Deep Networks



- Are these networks trainable ?
 - Advances in computation and processing
 - **Graphical processing units** (GPUs) performing multiple parallel multiply accumulate operations.
 - Large amounts of supervised data sets